

Reporte de Práctica — Paradigmas de Ejecución

Control de múltiples LEDs: delay() vs millis()

Sistemas Programables | Benjamín Salas Vizcarra | Septiembre 2026

Parte 1 Antipatrón con delay()

Parte 2 Temporización no bloqueante con millis()

PARTE 1 — Antipatrón del delay() con Múltiples LEDs

1. Introducción

En esta primera parte de la práctica se implementó el control de tres LEDs utilizando la función **delay()** del entorno Arduino. El objetivo fue observar y documentar el comportamiento bloqueante que esta función produce cuando se intenta manejar múltiples tareas con distintos intervalos de tiempo de forma aparentemente simultánea.

2. Diagrama del circuito

El circuito consiste en tres LEDs conectados a los pines digitales del Arduino UNO R4 WiFi, cada uno con su resistencia de 220Ω en serie para limitar la corriente.

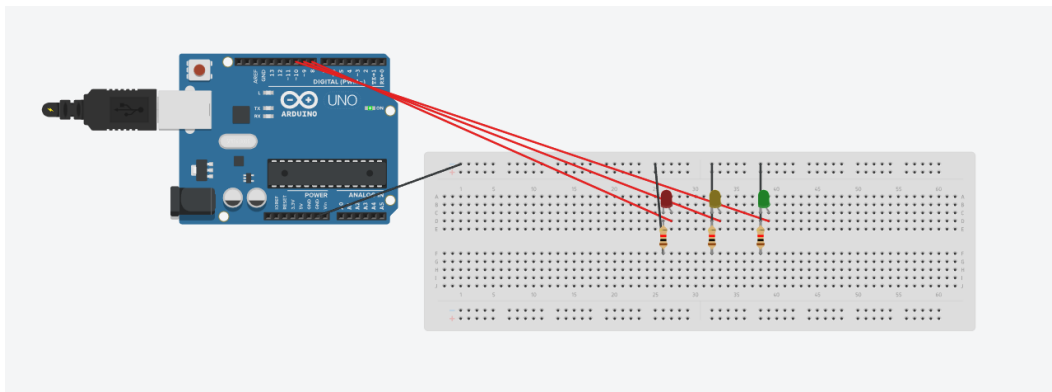


Figura 1. Diagrama de conexiones del circuito con tres LEDs.

3. Evidencia del circuito físico

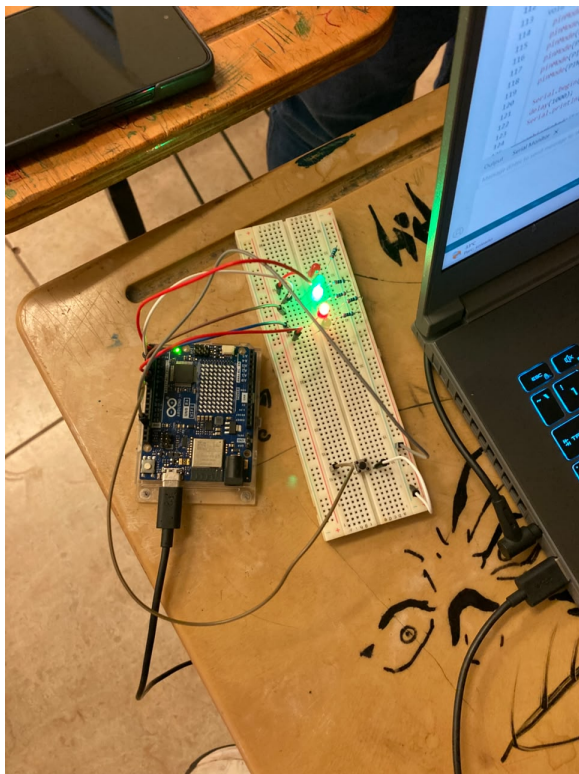


Figura 2. Circuito armado en protoboard con Arduino UNO R4 WiFi — Parte delay().

4. Comportamiento esperado vs. comportamiento real

Si cada LED funcionara de manera verdaderamente independiente, cada uno parpadearía con su propio periodo de forma simultánea. Sin embargo, el uso de **delay()** hace que el microcontrolador ejecute cada bloque de forma secuencial y bloqueante:

LED	Tiempo encendido	Tiempo apagado	Parpadeo esperado (independiente)
LED1	500 ms	500 ms	Cada 1000 ms (ciclo propio)
LED2	1000 ms	1000 ms	Cada 2000 ms (ciclo propio)
LED3	1500 ms	1500 ms	Cada 3000 ms (ciclo propio)

La tabla siguiente muestra el orden real de los eventos durante un ciclo completo:

Paso	Evento observado	Tiempo transcurrido
1	LED1 se enciende	0 ms
2	LED1 se apaga	500 ms
3	LED2 se enciende	1000 ms
4	LED2 se apaga	2000 ms
5	LED3 se enciende	3000 ms

Paso	Evento observado	Tiempo transcurrido
6	LED3 se apaga	4500 ms

5. Duración total del ciclo

Sumando los tiempos de encendido y apagado de los tres LEDs, un ciclo completo del loop() tarda:

$$500 + 500 + 1000 + 1000 + 1500 + 1500 = 6000 \text{ ms (6 segundos)}$$

Esto significa que LED1 no parpadea cada 500 ms como se pretendía, sino cada 6000 ms, esperando a que los otros dos LEDs terminen su propio ciclo.

6. Observaciones

- Los tres LEDs se encienden y apagan uno a la vez, nunca simultáneamente entre sí.
- El tiempo real de repetición de cada LED es la suma de todos los delay() del programa (6000 ms).
- Durante los delay(), el Arduino queda completamente inactivo para cualquier otra tarea.
- Visualmente, el patrón se percibe como una secuencia LED1 → LED2 → LED3, no como parpadeo independiente.

7. Video de evidencia

Grabación del circuito físico funcionando con delay(), donde se aprecia la secuencia bloqueante de los LEDs:

■ https://youtube.com/shorts/LCgG4536u_I

8. Conclusión — Parte 1

El uso de delay() para múltiples tiempos no produce parpadeos independientes, sino una cadena de esperas secuenciales acumuladas. Esto evidencia la necesidad de una estrategia no bloqueante para lograr que cada LED parpadee según su propio intervalo sin depender de los demás.

PARTE 2 — Temporización No Bloqueante con millis()

1. Introducción

En esta segunda parte se reimplementó el mismo circuito de tres LEDs utilizando la función **millis()** para la gestión del tiempo. Este enfoque no bloqueante permite que cada LED controle su propio intervalo de parpadeo de forma verdaderamente independiente, manteniendo el microcontrolador siempre disponible para otras tareas.

2. Diagrama del circuito

El mismo circuito de la Parte 1 fue reutilizado, añadiendo la tarea del mensaje serial al Monitor Serial cada 3000 ms como cuarta tarea concurrente.

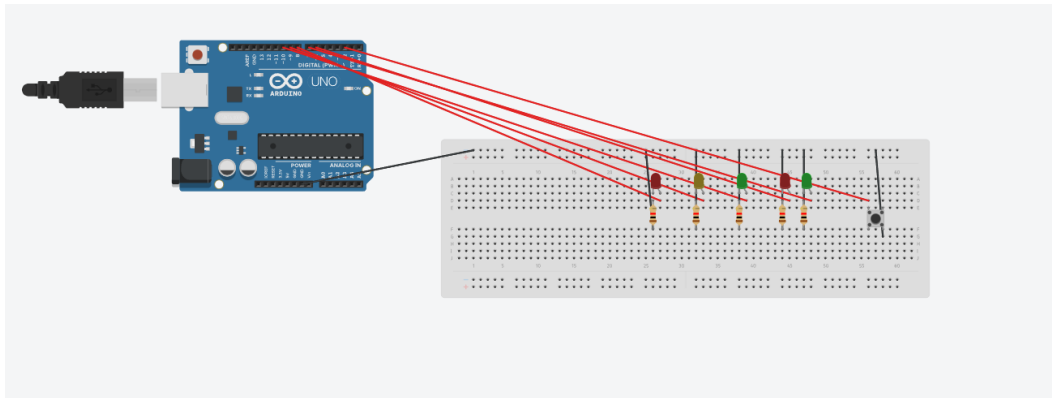


Figura 3. Diagrama de conexiones para la Parte 2 con millis() y tarea serial.

3. Evidencia del circuito físico

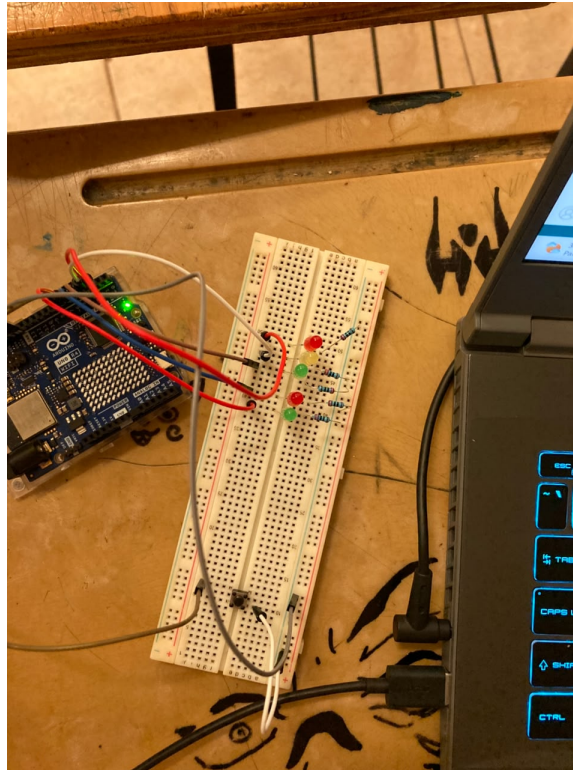


Figura 4. Circuito en funcionamiento — los LEDs parpadeando simultáneamente con `millis()`.

4. Comportamiento observado por tarea

A diferencia de la Parte 1, cada una de las cuatro tareas se ejecutó de forma independiente sin bloquear a las demás:

Tarea	Intervalo	Comportamiento observado	Independencia
LED1	500 ms	Parpadea de forma continua e independiente	Sin interferencia de LED2 ni LED3
LED2	1000 ms	Parpadea de forma continua e independiente	Sin interferencia de LED1 ni LED3
LED3	1500 ms	Parpadea de forma continua e independiente	Sin interferencia de LED1 ni LED2
Serial	3000 ms	Imprime "Hola Mundo" en el Monitor Serial	No afecta el parpadeo de los LEDs

5. Evidencia del Monitor Serial

Captura del Monitor Serial mostrando la impresión periódica de "Hola Mundo" cada 3 segundos, ocurriendo en paralelo mientras los tres LEDs seguían parpadeando en el circuito físico:

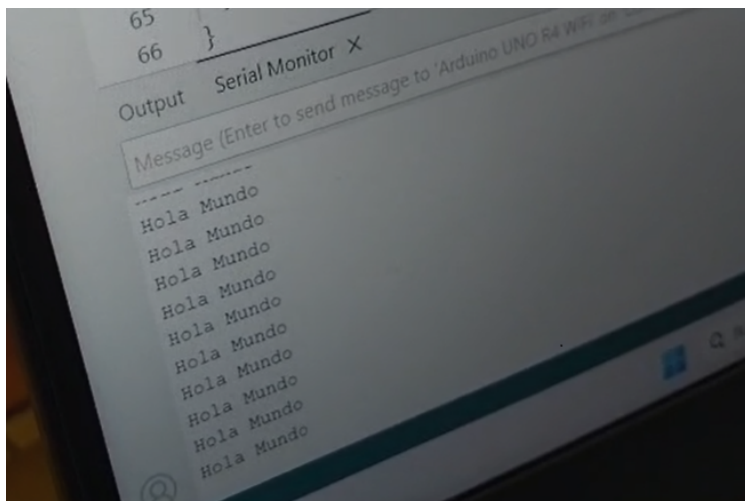


Figura 5. Monitor Serial mostrando "Hola Mundo" cada 3000 ms de forma no bloqueante.

6. Comparación de ciclos: delay() vs. millis()

En la Parte 1 un ciclo completo tardaba 6000 ms. En la Parte 2 no existe un 'ciclo completo' en ese sentido: el loop() corre miles de veces por segundo y cada tarea se dispara en cuanto cumple su propio intervalo:

Aspecto	delay()	millis()
Tipo de ejecución	Secuencial	Concurrente
Ciclo real de LED1	6000 ms	500 ms
Ciclo real de LED2	6000 ms	1000 ms
Ciclo real de LED3	6000 ms	1500 ms
Respuesta a entradas	Bloqueada	Siempre activa
Escalabilidad	Baja	Alta

7. Observaciones generales

- Los tres LEDs mostraron parpadeo simultáneo, cada uno a su propia velocidad, imposible con delay().
- El mensaje "Hola Mundo" se imprimió cada 3 segundos sin retrasos ni saltos, aun mientras los LEDs cambiaban.
- No se observó ningún bloqueo perceptible: el sistema respondió de forma fluida durante toda la prueba.
- Agregar la tarea serial no requirió modificar la lógica de los LEDs, confirmando la escalabilidad del patrón.

8. Video de evidencia

Grabación del circuito funcionando con millis(), mostrando los tres LEDs parpadeando simultáneamente:

■ <https://youtube.com/shorts/VZnIhfNHM9s>

9. Conclusión — Parte 2

La temporización basada en `millis()` logra el comportamiento originalmente buscado: múltiples tareas con distintos intervalos ejecutándose de forma verdaderamente concurrente. La comparación directa entre ambas partes evidencia la superioridad de este enfoque no bloqueante para cualquier proyecto que requiera manejar más de una temporización al mismo tiempo.